LTS Template Project Guide

This is a short guide on how to use the various aspects of this template project! The idea is to give you the base tools that MOST games need, so you can focus more on making the game rather than the core systems.

The template includes:

- Game Manager
    - Game States
    - Time Scale Setting
- Sound Manager
    - Volume Controls
    - Volume Persistence
    - Audio Library
- Input Manager
    - Setting New Inputs
    - Press, Hold, Release Checking
    - Preset Action Maps
- Scene Manager
    - Scene Input Field
    - Scene Fade Transitions
    - Loading Bar
    - Input Blocking
- Main menu / Pause Menu
    - Mouse, Keyboard & Controller Support
    - Options
- LTS_Organise
    - Colour Code Hierarchy and Project
    - Assign Icons

Game Manager

CurrentState tracks one of three enum values: LoadingScene, Playing, or Paused.

Pause() sets Time.timeScale = 0 and changes the state to Paused.
Resume() restores Time.timeScale = 1 and switches back to Playing.

```
2 references
public static void Pause() => Mgr.Pause();
3 references
public static void Resume() => Mgr.Resume();
```

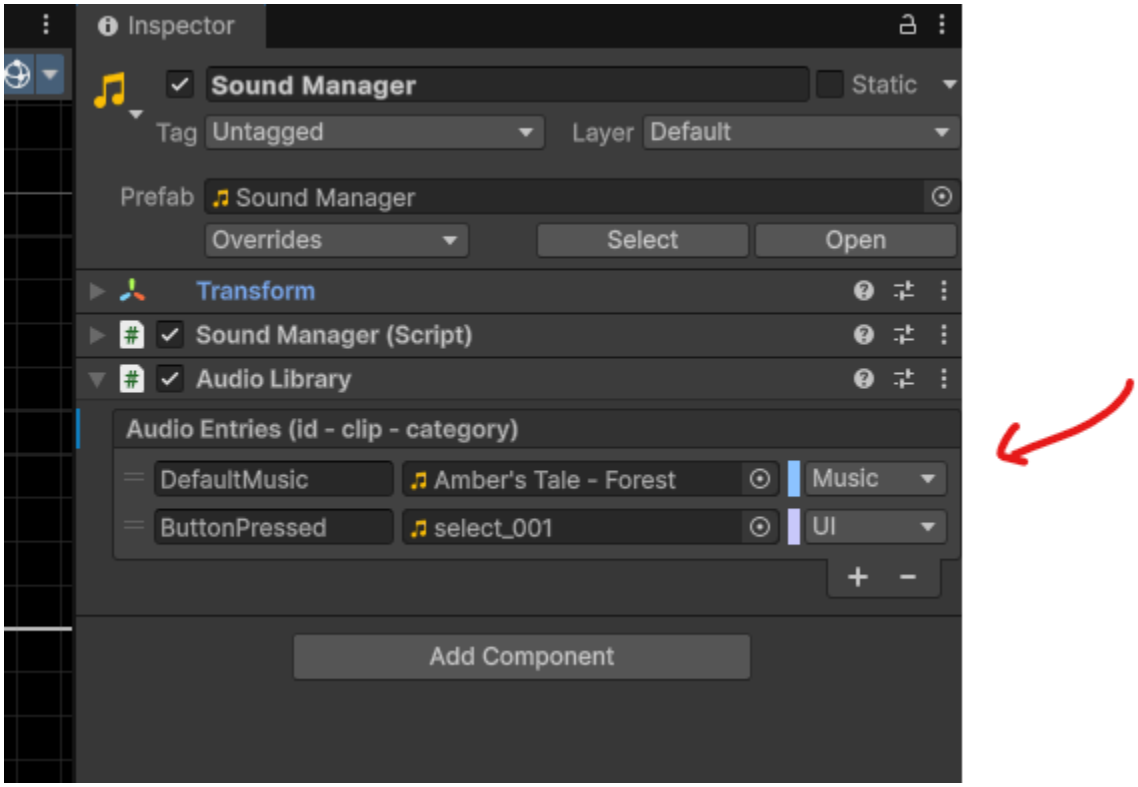
Call Pause and Resume from an external script using GameManager.Pause(); and GameManager.Resume(); as seen in the PauseMenu Script below.

```
1 reference
private void OnPause()
{
    Sound.PlaySound("ButtonPressed", 1f, 0.3f);
    GameManager.Pause();

    Input.SetMap(ActionMap.UI);
    EventSystem.current.firstSelectedGameObject = resumeButton.gameObject;

    pauseMenuContainer.SetActive(true);
}
```

Sound Manager

Each AudioCategory maps to a dedicated AudioMixerGroup (Music, SFX, UI, Ambience, Master). Global volume sliders are used in Main Menu and Pause Menu and audio levels persist between sessions.

Directly reference an AudioSource from your scripts or add sounds to the AudioLibrary. Tag each sound with the MixerGroup type for volume controls automatically.



Play Sounds using: Sound.PlaySound("SoundID"); if referenced in the AudioLibrary.

```
2 references
public AudioSource PlaySound(string id, float volume = 1f, float pitchVar = 0f, bool loop = false, bool stopWithFade = false, float fadeOut = 0.5f)
{
```

Optionally set volume, pitch variation, looping, fade-out:

Play Music which automatically cross-fades between tracks: Sound.PlayMusic("MainTheme");
Optionally set volume and crossfade duration

```
2 references
public void PlayMusic(string id, float tgtVol = 1f, float fade = 1f)
{
```

Input Manager

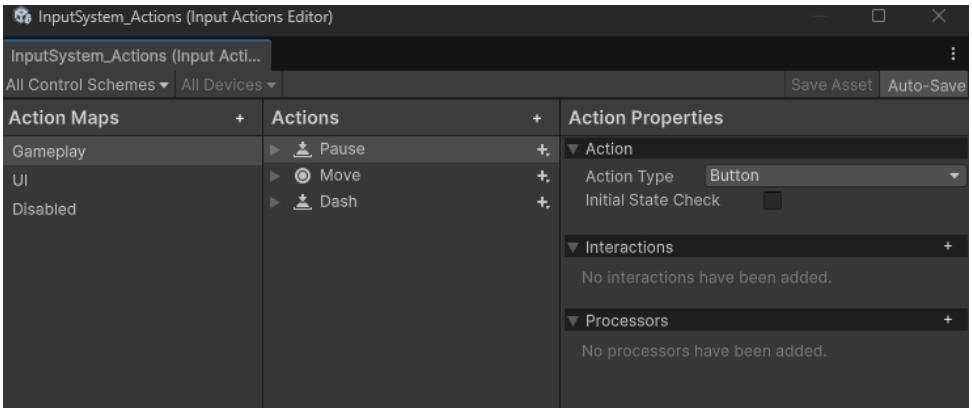
A centralized, easy-to-use system built on Unity's "new" Input System.

Quickly poll input states with the static Input class:

```
if (Input.Pressed(GameAction.Jump)) { /* Jump logic */ }
if (Input.Held(GameAction.Dash, holdTimeOverride: 0.5f)) { /* Charge Dash */ }
if (Input.Released(GameAction.Dash)) { /* Charge Dash after release */ }
```

Easily toggle between input contexts (e.g., Gameplay/UI/Disabled):
`Input.SetMap(ActionMap.Gameplay);`

Action Maps will only allow isolation of Input sets. For example in the pause menu we set the action map to UI to prevent player input from the gameplay action map.




```
1 reference
private void OnPause()
{
    Sound.PlaySound("ButtonPressed", 1f, 0.3f);
    GameManager.Pause();

    Input.SetMap(ActionMap.UI);
    EventSystem.current.firstSelectedGameObject = resumeButton.gameObject;

    pauseMenuContainer.SetActive(true);
}

2 references
private void OnResume()
{
    Sound.PlaySound("ButtonPressed", 1f, 0.3f);
    GameManager.Resume();

    Input.SetMap(ActionMap.Gameplay);

    pauseMenuContainer.SetActive(false);
    optionsMenuContainer.SetActive(false);
}
```

Add new action names to your GameAction enum as they appear in the Actions section showcased in the above image.

Scripts - Input - GameAction

```
/// <summary>
/// List any action maps you would like. Only one can be active at a time.
/// </summary>
11 references
public enum ActionMap
{
    Gameplay,
    UI,
    Disabled
}

/// <summary>
/// List every gameplay action you want to poll here.
/// Make sure the action names in your .inputactions asset match these exactly.
/// </summary>
22 references
public enum GameAction
{
    //UI Action Map
    Resume,
    Navigate,
    Submit,
    Cancel,
    Point,
    Click,
    RightClick,

    //Gameplay Action Map
    Pause,
    Move,
    Jump,
    Dash,


}
```

Checklist:

- Create keybinds in the input action asset
- Add action to the game action enum
- Call new input and input method: if (Input.Pressed(GameAction.NEWACTION))

You can also read input values such as Move direction or Mouse Position Vector2.

Simply call as an example: Vector2 moveInput = Input.GetValue<Vector2>(GameAction.Move);

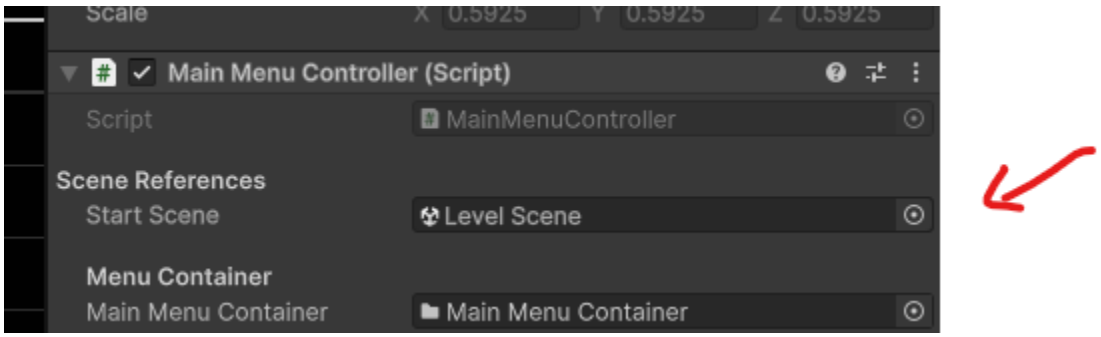
Scene Manager

A simple manager that handles scene fading, loading and includes a loading bar.

Call: SceneTransitionManager.LoadScene(SceneToLoad); for SceneField input
SceneTransitionManager.LoadScene("SceneToLoad"); for String input

```
{
    [Header("Scene References")]
    [Tooltip("Scene loaded when the Start button is pressed.")]
    [SerializeField] private SceneField startScene;
```

SceneField allows you to assign a Scene to the inspector.

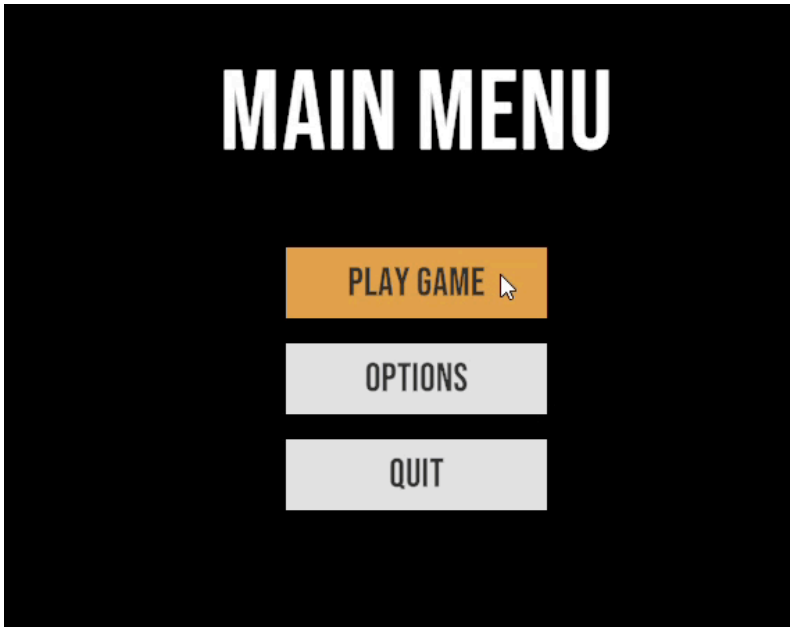


Note: During scene transitions both UI and Gameplay actionmaps are disabled so no inputs will be detected until the new scene is fully loaded.

Main menu / Pause Menu

```
mary>
(1 asset reference) | 0 references
ass MainMenuController : MenuBase
```

Both Main menu and Pause inherit from Menu base which just serves to maintain at least one UI element selected at all times regardless of if you switch from mouse to other input schemes.



Just ensure you call `RefreshSelection(defultSelectedButton.gameObject);` to set which gameobject should be first selected when the scene or panel loads.

```
1 reference
private void OnPause()
{
    Sound.PlaySound("ButtonPressed", 1f, 0.3f);
    GameManager.Pause();

    Input.SetMap(ActionMap.UI);
    RefreshSelection(resumeButton.gameObject);

    pauseMenuContainer.SetActive(true);
}
```

The pause menu is a prefab that can just be dropped into any scene and work out of the box.

LTS Organise

This is a super lightweight and easy to use asset!

To get started simply press ALT + Left Mouse, on any GameObject in the hierarchy or Folder in the Project tab. You are then able to select a colour and associated icon to be displayed.

If you would like to add additional colours or icons you can add them inside PaletteData.cs.